

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 6

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Named Pipes FIFO Server-Client Application

Question:

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Source Code:

Server Program

```

Server finished.
haasini@SSH:~/OSSP_upload/Practical_Week6$ cat > P6_T1_server_multi.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>

#define FIFO_NAME "ossp_fifo"

int main(void)
{
    char buffer[256];

    unlink(FIFO_NAME);

    if (mkfifo(FIFO_NAME, 0666) == -1)
    {
        perror("mkfifo");
        return 1;
    }

    printf("Server started.\n");
    printf("Waiting for multiple client messages...\n");

    int fd = open(FIFO_NAME, O_RDONLY);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    int count = 0;

    while (count < 3)
    {
        ssize_t bytes = read(fd, buffer, sizeof(buffer) - 1);

        if (bytes > 0)
            count++;
    }

    gcc -Wall -Wextra P6_T1_server_multi.c -o P6_T1_server_multi
haasini@SSH:~/OSSP_upload/Practical_Week6$

```

Client Program

```
EOF return 0; // client finished.\n", %s\n", buffer, ei) < 1)) > 0)\nhaasini@SSH:~/OSSP_upload/Practical_Week6$ cat > P6_T1_client.c <<'EOF'\n#include <stdio.h>\n#include <stdlib.h>\n#include <unistd.h>\n#include <fcntl.h>\n#include <string.h>\n\ndefine FIFO_NAME "ossp_fifo"\n\nint main(int argc, char *argv[])\n{\n    if (argc != 2)\n    {\n        printf("Usage: %s \\\"message\\\"\\n", argv[0]);\n        return 1;\n    }\n\n    int fd = open(FIFO_NAME, O_WRONLY);\n\n    if (fd == -1)\n    {\n        perror("open");\n        return 1;\n    }\n\n    write(fd, argv[1], strlen(argv[1]));\n\n    printf("Client PID %d sent: %s\\n", getpid(), argv[1]);\n\nEOF return 0;;\nhaasini@SSH:~/OSSP_upload/Practical_Week6$ gcc -Wall -Wextra P6_T1_server.c\n-o P6_T1_server\ngcc -Wall -Wextra P6_T1_client.c -o P6_T1_client
```

Output:

Server-Side Multiple Client Communication

```
haasini@SSH:~/OSSP_upload/Practical_Week6$ cd ~/OSSP_upload/Practical_Week6
./P6_T1_server_multi
Server started.
Waiting for multiple client messages...
Server received: Message from Client 2
Server received: Message from Client 1
Server received: Message from Client 3
Server received all 3 client messages.
haasini@SSH:~/OSSP_upload/Practical_Week6$ |
```

Client-Side Multiple Client Communication

```
[3] Done
haasini@SSH:~/OSSP_upload/Practical_Week6$ cd ~/OSSP_upload/Practical_Week6
./P6_T1_client "Message from Client 1" &
./P6_T1_client "Message from Client 2" &
./P6_T1_client "Message from Client 3" &
wait
[1] 802
[2] 803
Client PID 803 sent: Message from Client 2
Client PID 802 sent: Message from Client 1
[3] 804
[1] Done ./P6_T1_client "Message from Client 1"
[2]- Done ./P6_T1_client "Message from Client 2"
Client PID 804 sent: Message from Client 3
```

Observations:

1. mkfifo() creates named FIFO special files in the filesystem.
2. The server and clients are separate processes that communicate through the FIFO.
3. Multiple clients send requests to the same server FIFO (ossp_fifo).
4. Each client includes its process ID so the server can identify the sender.
5. The server reads and processes requests from the shared FIFO buffer.
6. Each client writes its message to the same named FIFO, allowing the server to receive messages from multiple client processes.
7. The server successfully reads messages sent by three different client processes through the shared FIFO.
8. Multiple clients may execute concurrently, so the order of messages depends on process scheduling.
9. FIFO communication allows inter-process communication (IPC) between processes that do not need to share a parent-child relationship.
10. The actual output confirms that messages from three concurrently executed clients are successfully received by the server.

Task 2: POSIX Signal Handling Program

Question:

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Source Code:

```
haasini@SSH:~/OSSP_upload/Practical_Week6$ cat > P6_T2.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
        printf("\nReceived SIGINT (Interrupt signal)\n");

    else if (sig == SIGUSR1)
        printf("\nReceived SIGUSR1 (User-defined signal)\n");

    else if (sig == SIGTERM)
    {
        printf("\nReceived SIGTERM (Termination signal)\n");
        printf("Program terminating safely.\n");
        exit(0);
    }
}

int main(void)
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handling program started.\n");
    printf("PID: %d\n", getpid());
    printf("Waiting for SIGINT, SIGUSR1 and SIGTERM...\n");

    while (1)
        pause();

    return 0;
}
EOF
```

Output:

```
gcc -Wall -Wextra P6_T2.c -o P6_T2
./P6_T2
Signal handling program started.
PID: 812
Waiting for SIGINT, SIGUSR1 and SIGTERM...

Received SIGUSR1 (User-defined signal)

Received SIGINT (Interrupt signal)

Received SIGTERM (Termination signal)
Program terminating safely.
haasini@SSH:~/OSSP_upload/Practical_Week6$ |
```

```
haasini@SSH:~/OSSP_upload/Practical_Week6$ kill -SIGUSR1 812
haasini@SSH:~/OSSP_upload/Practical_Week6$ kill -SIGINT 812
haasini@SSH:~/OSSP_upload/Practical_Week6$ kill -SIGTERM 812
```

Observations:

1. signal() registers signal handlers for SIGINT, SIGTERM, and SIGUSR1 signals.
2. SIGUSR1 is a user-defined signal that can be sent by the kill command.
3. SIGINT represents the interrupt signal (typically triggered by Ctrl+C).
4. SIGTERM requests termination of the process and allows graceful shutdown.
5. Signals are asynchronous events that interrupt normal program flow.
6. The signal handler function executes when the corresponding signal arrives at the process.
7. pause() blocks the process and keeps it waiting for signals.
8. kill -SIGUSR1, kill -SIGINT, and kill -SIGTERM send the respective signals to the target process by PID.
9. SIGTERM is handled for graceful shutdown, printing a termination message before calling exit(0).
10. The program successfully demonstrates handling of all three signals in the correct sequence.